

TP6 - DEEP LEARNING ET MLOPS : CLASSIFICATION D'IMAGES MÉDICALES



CONTEXTE MÉTIER

Vous êtes Data Scientist dans un service de radiologie hospitalière. L'objectif est de développer un système d'aide au diagnostic automatique pour détecter la pneumonie à partir de radiographies thoraciques (Chest X-Ray).

Enjeu médical : La pneumonie tue plus de 2 millions d'enfants par an dans le monde. Un diagnostic précoce et précis est crucial.

Objectif technique : Créer un pipeline complet de Deep Learning, du développement du modèle CNN jusqu'au déploiement en production avec MLOps.

DATASET

Source : Kaggle - Chest X-Ray Images (Pneumonia)

Lien : <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

Caractéristiques :

- 5,863 images JPEG de radiographies thoraciques
- 2 classes : NORMAL / PNEUMONIA
- Résolution : 1024x1024 pixels (variable)
- Train : 5,216 images
- Validation : 16 images
- Test : 624 images

Téléchargement :

```
```bash
cd TD/data
python download_data.py
```
```

Ou manuellement depuis :

<https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>

OBJECTIFS PÉDAGOGIQUES

À l'issue de ce TP, vous serez capable de :

1. Construire et entraîner un CNN from scratch pour la classification d'images
2. Implémenter le Transfer Learning avec des architectures modernes (ResNet50, EfficientNet)
3. Optimiser les hyperparamètres et gérer l'overfitting
4. Rendre le modèle explicable avec Grad-CAM (et SHAP en bonus)
5. Déployer un modèle via une API REST (FastAPI)
6. Conteneuriser avec Docker (CPU ou GPU)
7. Monitorer les performances en production

PARTIE 1 : EXPLORATION ET PRÉPARATION DES DONNÉES (1h30)

Mission 1.1 - Exploration du Dataset

Notebook : `01\_EDA\_Deep\_Learning.ipynb`

Tâches :

1. Charger et visualiser :

- Afficher 10 images de chaque classe (NORMAL vs PNEUMONIA)
- Analyser la distribution des classes (déséquilibre ?)
- Vérifier les dimensions des images

2. Statistiques descriptives :

- Taille moyenne des images
- Distribution des intensités de pixels
- Identifier d'éventuelles anomalies (images corrompues, trop sombres, etc.)

3. Insights métier :

- Différences visuelles entre radiographies normales et pneumonie
- Zones d'intérêt (poumons, opacités)

Livrable :

- Rapport d'exploration (3-5 visualisations + analyse)

-

Mission 1.2 - Data Augmentation

Objectif : Augmenter artificiellement la taille du dataset pour réduire l'overfitting.

Techniques à implémenter :

```
```python
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
train_datagen = ImageDataGenerator(
 rescale=1./255,
 rotation_range=20, Rotation aléatoire $\pm 20^\circ$
 width_shift_range=0.2, Décalage horizontal
 height_shift_range=0.2, Décalage vertical
 shear_range=0.2, Cisaillement
 zoom_range=0.2, Zoom aléatoire
 horizontal_flip=True, Flip horizontal
 fill_mode='nearest'
```

```
)
```

```
```
```

Principe mathématique :

- Rotation : Matrice de rotation $R(\theta)$
- Translation : Décalage (tx, ty)
- Zoom : Facteur d'échelle s

Livrable :

- Pipeline de data augmentation fonctionnel
- Visualisation de 5 images augmentées

-

PARTIE 2 : CNN FROM SCRATCH (2h)

Mission 2.1 - Architecture CNN Personnalisée

Notebook : `02\_CNN\_From\_Scratch.ipynb`

Objectif :

Créer un CNN from scratch avec TensorFlow/Keras.

Architecture recommandée :

```
```
```

INPUT (224x224x3)

↓

CONV2D (32 filters, 3x3) + ReLU

↓

MAXPOOLING (2x2)

↓  
 CONV2D (64 filters, 3x3) + ReLU  
 ↓  
 MAXPOOLING (2x2)  
 ↓  
 CONV2D (128 filters, 3x3) + ReLU  
 ↓  
 MAXPOOLING (2x2)  
 ↓  
 FLATTEN  
 ↓  
 DENSE (128 neurons) + ReLU + Dropout(0.5)  
 ↓  
 DENSE (1 neuron) + Sigmoid  
 ...

Fonctions d'activation :

- ReLU :  $f(x) = \max(0, x)$  → Non-linéarité, résout le vanishing gradient
- Sigmoid :  $\sigma(x) = 1 / (1 + e^{(-x)})$  → Output entre 0 et 1 pour classification binaire

Convolution 2D :

Formule :  $\text{Output} = (\text{Input} \times \text{Kernel}) + \text{Bias}$

Pour chaque filtre, la convolution calcule :

...

$$Y[i,j] = \sum \sum X[i+m, j+n] \times K[m,n] + b$$

...

### **Tâches :**

1. Créer le modèle avec Keras Sequential ou Functional API
2. Compiler :
  - Loss : ``binary_crossentropy``
  - Optimizer : ``Adam(learning_rate=0.001)``
  - Metrics : ``accuracy``, ``AUC``, ``precision``, ``recall``
3. Entraîner :
  - Epochs : 20-30
  - Batch size : 32
  - Callbacks : EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
4. Évaluer :
  - Matrice de confusion
  - Courbes d'apprentissage (loss, accuracy)
  - ROC-AUC

Livrables :

- Modèle CNN entraîné (`` .h5`` ou `` .keras``)
- Rapport de performances (accuracy, precision, recall, AUC)
- Graphiques (loss curves, confusion matrix)

Performance attendue :

- Accuracy : 70-80% (baseline acceptable pour from scratch)

-

### **Mission 2.2 - Régularisation et Optimisation**

Objectif : Réduire l'overfitting.

Techniques à tester :

1. Dropout :
  - Principe : Désactiver aléatoirement p% des neurones pendant l'entraînement
  - Formule : ``y = Dropout(x, p)`` avec probabilité p (ex: 0.5)

- Tester  $p = \{0.3, 0.5, 0.7\}$
  - 2. Batch Normalization :
    - Normaliser les activations de chaque couche
    - Formule :  $\text{BN}(x) = \gamma \times (x - \mu) / \sigma + \beta$
    - Accélère la convergence, réduit l'overfitting
  - 3. L2 Regularization :
    - Pénaliser les poids trop élevés
    - Loss totale :  $L_{\text{total}} = L_{\text{data}} + \lambda \times \sum w^2$
  - 4. Learning Rate Schedule :
    - Réduire le learning rate progressivement
    - Callback : `ReduceLROnPlateau(factor=0.5, patience=3)`
- Livrable :
- Tableau comparatif des stratégies (accuracy train vs validation)
  - Modèle optimisé avec meilleure généralisation

### **PARTIE 3 : TRANSFER LEARNING (1h30)**

#### **Mission 3.1 - Fine-Tuning avec ResNet50**

Notebook : ``03_Transfer_Learning.ipynb``

Principe du Transfer Learning :

1. Charger un modèle pré-entraîné (ImageNet avec 1000 classes)
2. Geler les couches de base (feature extraction)
3. Ajouter des couches personnalisées pour notre tâche (2 classes)
4. Fine-tuner les dernières couches si nécessaire

Architecture :

...

ResNet50 (pré-entraîné ImageNet)

- Couches gelées : 0 à -20
- Couches entraînables : -20 à -1

↓

GlobalAveragePooling2D

↓

Dense(256, activation='relu')

↓

Dropout(0.5)

↓

Dense(1, activation='sigmoid')

...

Code template :

```

'''python
from tensorflow.keras.applications import ResNet50
from tensorflow.keras import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
Charger ResNet50 sans la couche de classification
base_model = ResNet50(
 weights='imagenet',
 include_top=False,
 input_shape=(224, 224, 3)
)
Geler les couches de base
base_model.trainable = False

```

*Ajouter les couches personnalisées*

```
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
predictions = Dense(1, activation='sigmoid')(x)
model = Model(inputs=base_model.input, outputs=predictions)
...
```

Tâches :

1. Entraîner avec couches gelées (5-10 epochs)
2. Dégeler les dernières couches et fine-tuner (10 epochs supplémentaires avec LR réduit)
3. Comparer :
  - ResNet50
  - EfficientNetB0 (optionnel)
  - VGG16 (optionnel)

Livrable :

- Modèle Transfer Learning entraîné
- Tableau comparatif : CNN from scratch vs Transfer Learning
- Justification du choix du meilleur modèle

Performance attendue :

- Accuracy : 90-95% (Transfer Learning doit nettement surpasser le from scratch)
- AUC :  $\geq 0.95$

-

#### **PARTIE 4 : EXPLICABILITÉ DU MODÈLE (1h)**

##### Mission 4.1 - Grad-CAM (Gradient-weighted Class Activation Mapping)

Notebook : `04\_Explicabilite\_SHAP\_GradCAM.ipynb`

Objectif :

Visualiser quelles zones de l'image le modèle utilise pour sa décision.

Principe mathématique Grad-CAM :

1. Calculer les gradients de la classe cible par rapport aux feature maps de la dernière couche Conv
2. Pondérer les feature maps par ces gradients
3. Sommer et appliquer ReLU pour obtenir une heatmap

Formule :

...

$L^c_{\text{Grad-CAM}} = \text{ReLU}(\sum \alpha_k^c \times A^k)$

...

où :

- $\alpha_k^c = (1/Z) \sum \sum (\partial y^c / \partial A^k_{ij})$  : Importance du filtre k pour la classe c
- $A^k$  : Feature map k de la dernière couche Conv

Code template :

```
```python
import tensorflow as tf
import numpy as np
import cv2

def make_gradcam_heatmap(img_array, model, last_conv_layer_name, pred_index=None):
    Créer un modèle qui mappe l'input aux activations de la dernière conv + output
    grad_model = tf.keras.models.Model(
        [model.inputs],
        [model.get_layer(last_conv_layer_name).output, model.output]
    )
```

```

with tf.GradientTape() as tape:
    last_conv_layer_output, preds = grad_model(img_array)
    if pred_index is None:
        pred_index = tf.argmax(preds[0])
    class_channel = preds[:, pred_index]

```

Gradients de la classe prédite par rapport aux feature maps
 grads = tape.gradient(class_channel, last_conv_layer_output)

Moyenne des gradients (importance de chaque filtre)
 pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))

Pondérer les feature maps par les gradients
 last_conv_layer_output = last_conv_layer_output[0]
 heatmap = last_conv_layer_output @ pooled_grads[..., tf.newaxis]
 heatmap = tf.squeeze(heatmap)

Normaliser entre 0 et 1
 heatmap = tf.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
 return heatmap.numpy()

...

Tâches :

1. Implémenter Grad-CAM pour 5 images NORMAL et 5 images PNEUMONIA
2. Superposer les heatmaps sur les radiographies originales
3. Analyser : Le modèle se concentre-t-il sur les poumons (zones pertinentes) ?

Livrable :

- 10 visualisations Grad-CAM
- Analyse qualitative de l'interprétabilité
-

Mission 4.2 - SHAP pour Deep Learning (OPTIONNEL - BONUS +0.5 pt)

Objectif : Expliquer l'importance de chaque pixel pour la prédiction.

Note : SHAP est optionnel car très lent à exécuter (10-30 min). Si vous manquez de temps, concentrez-vous sur Grad-CAM qui est plus rapide et tout aussi pertinent pour l'explicabilité médicale.

Code template (si vous faites le bonus) :

```

```python
import shap

Charger le modèle
model = tf.keras.models.load_model('models/best_model.h5')
Créer l'explainer SHAP (DeepExplainer pour réseaux de neurones)
background = X_train[:50] Échantillon de référence (réduit pour rapidité)
explainer = shap.DeepExplainer(model, background)
Calculer SHAP values pour SEULEMENT 3-5 images (sinon trop long)
shap_values = explainer.shap_values(X_test[:3])
Visualiser
shap.image_plot(shap_values, X_test[:3])
```

```

Livrable (si bonus) :

- 3-5 visualisations SHAP
- Comparaison Grad-CAM vs SHAP (similarités/différences)

Conseil : Si SHAP prend >15 min, passez à la partie suivante et revenez-y si vous avez du temps en fin de TP.

PARTIE 5 : DÉPLOIEMENT ET MLOPS (2h)

Mission 5.1 - API REST avec FastAPI

Objectif : Créer une API pour servir le modèle en production.

Fichier : `TD/api/app.py`

Architecture API :

...

POST /predict

- Input: Image (fichier uploadé ou base64)
- Output: {"class": "PNEUMONIA", "probability": 0.92, "confidence": "High"}

GET /health

- Output: {"status": "healthy", "model_loaded": true}

GET /model/info

- Output: {"model_type": "ResNet50", "accuracy": 0.94, "version": "1.0"}

...

Code template :

```
```python
```

```
from fastapi import FastAPI, File, UploadFile
from fastapi.responses import JSONResponse
import tensorflow as tf
import numpy as np
from PIL import Image
import io

app = FastAPI(title="Pneumonia Detection API")
Charger le modèle au démarrage
model = tf.keras.models.load_model('../models/best_model.h5')
@app.post("/predict")
async def predict(file: UploadFile = File(...)):
 Lire l'image
 contents = await file.read()
 image = Image.open(io.BytesIO(contents))
```

Preprocessing

```
image = image.resize((224, 224))
image = image.convert('RGB')
img_array = np.array(image) / 255.0
img_array = np.expand_dims(img_array, axis=0)
```

Prédiction

```
prediction = model.predict(img_array)[0][0]
```

Interprétation

```
class_label = "PNEUMONIA" if prediction > 0.5 else "NORMAL"
confidence = "High" if (prediction > 0.8 or prediction < 0.2) else "Medium"
```



```

 return {
 "class": class_label,
 "probability": float(prediction),
 "confidence": confidence
 }
@app.get("/health")
def health():
 return {"status": "healthy", "model_loaded": model is not None}
@app.get("/model/info")
def model_info():
 return {
 "model_type": "ResNet50 Transfer Learning",
 "input_shape": [224, 224, 3],
 "classes": ["NORMAL", "PNEUMONIA"],
 "version": "1.0"
 }
...

```

#### Tâches :

1. Compléter l'API avec les endpoints ci-dessus
2. Tester localement avec `uvicorn app:app reload`
3. Tester avec `curl` ou Postman

Livrable :

- API fonctionnelle
- Documentation des endpoints (README\_API.md)

#### Mission 5.2 - Docker (Version Simplifiée CPU)

Objectif : Conteneuriser l'API pour le déploiement.

Fichiers fournis : `TD/docker/Dockerfile` et `TD/docker/docker-compose.yml` sont déjà créés pour vous !

Note : Pour simplifier, nous utilisons la version CPU (pas besoin de nvidia-docker).

Si vous avez un GPU NVIDIA (**Google Colab**) et voulez l'utiliser, voir les instructions dans `TD/docker/README\_DOCKER.md`.

Dockerfile fourni (version CPU) :

```

```dockerfile
Image de base TensorFlow CPU (plus simple, pas de GPU requis)
FROM tensorflow/tensorflow:2.18.0
WORKDIR /app
Installer dépendances système
RUN apt-get update && apt-get upgrade -y && \
    apt-get install -y curl && \
    apt-get clean && rm -rf /var/lib/apt/lists/
Copier requirements et installer
COPY requirements.txt .
RUN pip install no-cache-dir upgrade pip && \
    pip install no-cache-dir -r requirements.txt
Copier l'application
COPY api/ ./api/
COPY models/ ./models/

```

```

Créer utilisateur non-root (sécurité)
RUN useradd -m -u 1000 appuser && \
    chown -R appuser:appuser /app
USER appuser
EXPOSE 8000
HEALTHCHECK interval=30s timeout=5s start-period=15s retries=3 \
    CMD curl fail http://localhost:8000/health || exit 1
CMD ["uvicorn", "api.app:app", "host", "0.0.0.0", "port", "8000"]
...

```

docker-compose.yml fourni :

```

...yaml
services:
  pneumonia-api:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    container_name: pneumonia-api
    ports:
      - "8000:8000"
    volumes:
      - ../models:/app/models:ro
    restart: unless-stopped
    mem_limit: 2g
    cpus: 1.0
...

```

Tâches :

1. Vérifier que les fichiers Docker sont présents dans `TD/docker/`
2. Build : `cd TD/docker && docker build -t pneumonia-api:latest ..`
3. Run : `docker-compose up -d`
4. Tester l'API conteneurisée

Livrable :

- Dockerfile fonctionnel
- docker-compose.yml
- Guide de déploiement

Mission 5.3 - Monitoring et Drift Detection

Objectif : Monitorer les performances du modèle en production.

Métriques à tracker :

1. Latence : Temps de réponse de l'API (objectif <500ms)
2. Distribution des prédictions : Ratio NORMAL/PNEUMONIA
3. Confiance : Distribution des probabilités
4. Drift : Changement de distribution des inputs (KL Divergence)

KL Divergence (Kullback-Leibler) : (<https://www.datacamp.com/tutorial/kl-divergence>)

...

$$D_{KL}(P || Q) = \sum P(x) \times \log(P(x) / Q(x))$$

...

- P : Distribution des données d'entraînement
- Q : Distribution des données en production
- Seuil d'alerte : $D_{KL} > 0.1$

Code template :

```

...python

```

```

import numpy as np
def kl_divergence(p, q, bins=50):
    """Calculer la KL divergence entre deux distributions."""
    p_hist, _ = np.histogram(p, bins=bins, density=True)
    q_hist, _ = np.histogram(q, bins=bins, density=True)

    Ajouter epsilon pour éviter log(0)
    p_hist = p_hist + 1e-10
    q_hist = q_hist + 1e-10

    kl_div = np.sum(p_hist * np.log(p_hist / q_hist))
    return kl_div

```

Livable :

- Script de monitoring (`src/monitoring.py`)
- Dashboard simple (optionnel : Streamlit ou Plotly Dash)
-

LIVRABLES FINAUX

1. Code et Notebooks (8 fichiers)
 - `01\_EDA\_Deep\_Learning.ipynb` (complété)
 - `02\_CNN\_From\_Scratch.ipynb` (complété)
 - `03\_Transfer\_Learning.ipynb` (complété)
 - `04\_Explicabilite\_SHAP\_GradCAM.ipynb` (complété)
 - `api/app.py` (API FastAPI)
 - `src/monitoring.py` (Monitoring)
 - `docker/Dockerfile` et `docker-compose.yml`
2. Modèles Sauvegardés (2 fichiers)
 - `models/cnn\_from\_scratch.h5`
 - `models/resnet50\_transfer\_learning.h5`
3. Rapport de Synthèse (Markdown ou PDF, 5-10 pages)

Structure :

1. Introduction et contexte médical
2. Exploration des données
3. Résultats CNN from scratch (architecture, performances)
4. Résultats Transfer Learning (comparaison ResNet50, EfficientNet)
5. Explicabilité (Grad-CAM, SHAP)
6. Déploiement MLOps (API, Docker)
7. Monitoring et Drift
8. Conclusion et recommandations
4. README Technique
 - Installation et setup
 - Instructions de lancement de l'API
 - Guide Docker
 - Exemples d'utilisation

Bonus :

- Comparaison de 3+ architectures (ResNet, EfficientNet, VGG)
- Dashboard de monitoring
- Tests unitaires de l'API
- Kubernetes deployment (optionnel)

-

RESSOURCES

Documentation Officielle

- [TensorFlow/Keras](https://www.tensorflow.org/api_docs)
- [FastAPI](https://fastapi.tiangolo.com/)
- [SHAP](https://shap.readthedocs.io/)
- [Docker](https://docs.docker.com/)

Architectures CNN

- [ResNet Paper](https://arxiv.org/abs/1512.03385)
 - [EfficientNet Paper](https://arxiv.org/abs/1905.11946)
- Grad-CAM
- [Grad-CAM Paper](https://arxiv.org/abs/1610.02391)

CONSEILS

1. Commencez simple : CNN from scratch basique avant d'optimiser
2. Utilisez GPU : Le Transfer Learning nécessite des ressources
3. Surveillez l'overfitting : Courbes de loss train vs validation
4. Testez l'API tôt : Ne pas attendre la fin pour l'intégration
5. Documentez : Commentaires dans le code, markdown dans les notebooks

Bon courage ! Ce TP représente l'aboutissement de vos compétences en Deep Learning et MLOps.